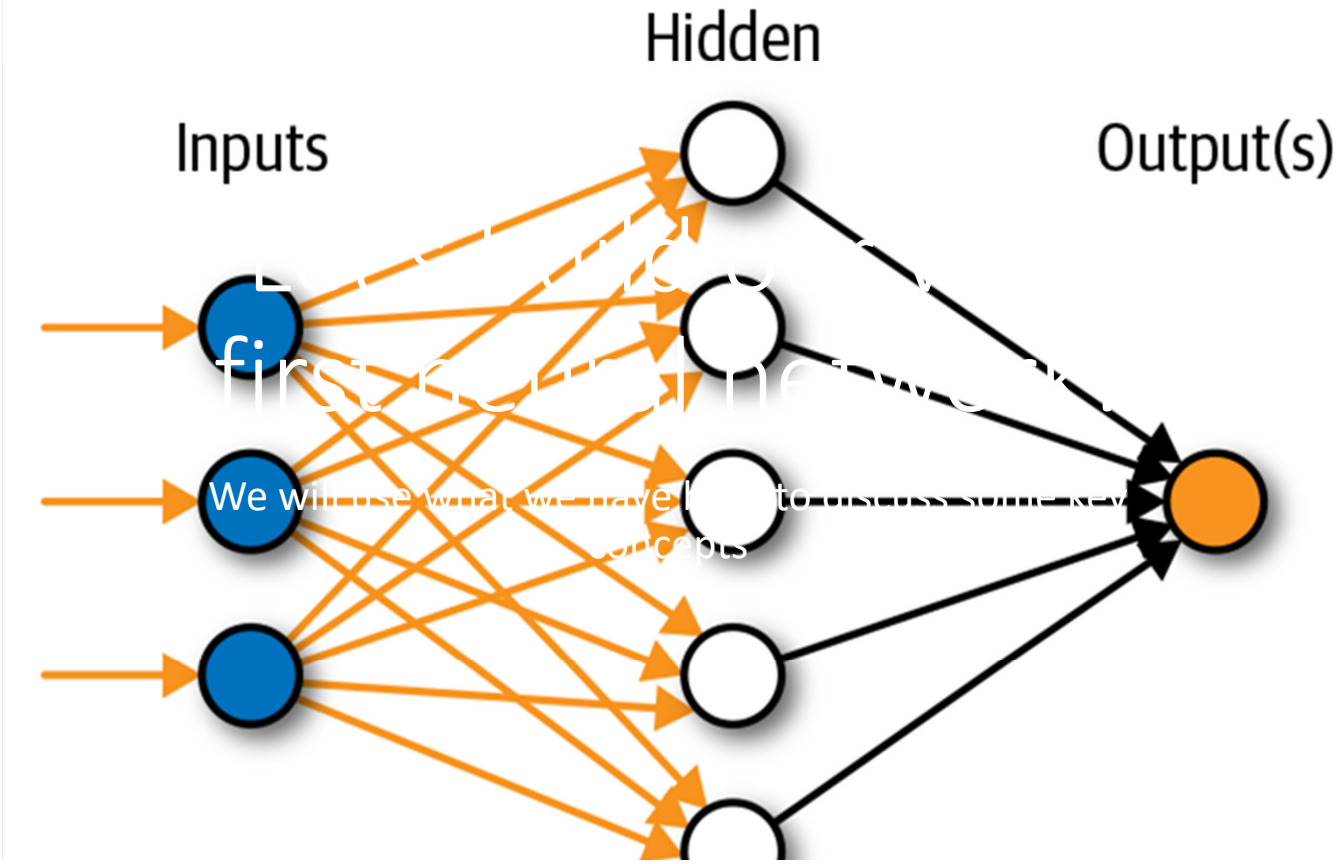
Artificial Neural Network



Platform

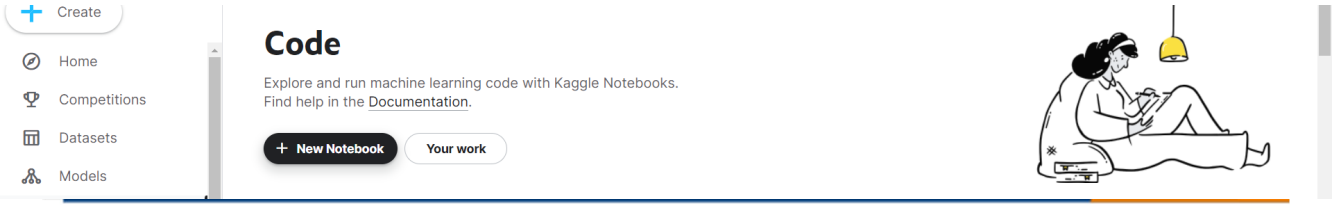
We will be using Python, and coding in Python on jupyter notebooks, hosted on Kaggle

Why Python? Well, it's the most popular data science language, and lots of ML (and deep learning) packages are coded in Python

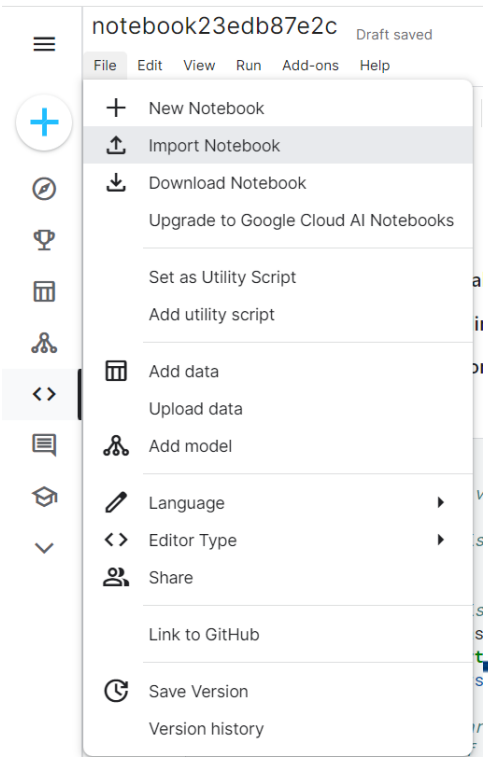
Why online jupyter notebooks? Coding environment allows us to integrate code and comments, and make use of GPUs

Introducing Kaggle notebooks

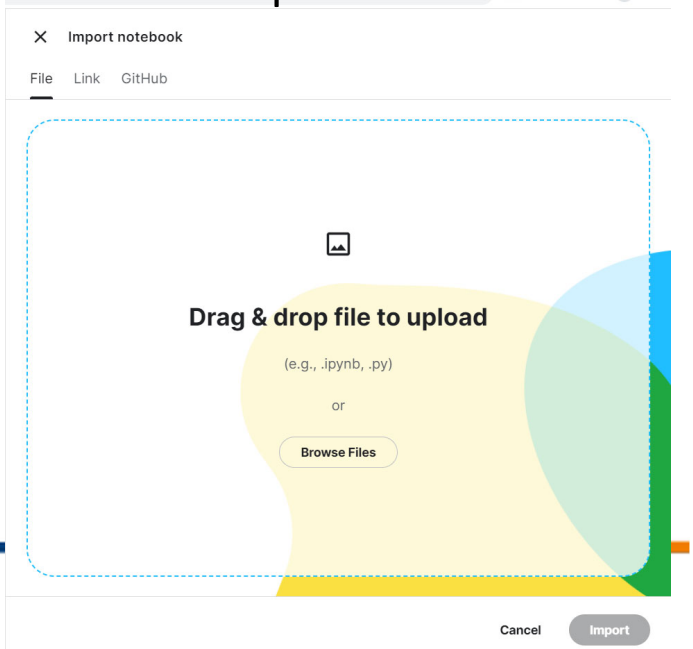
- Let's go to Kaggle and create a new notebook
- https://www.kaggle.com/code
  - Sign into your account (email + password)
- Select "New Notebook"



Go to "File" → "Import Notebook"



Drag and drop the "Class 1" notebook → Import



What we will notice…

- Code and text integrated together, along with the results of running the code → useful feature of a notebook
- Give your notebook a title, e.g. “RTCGPT-Class1”
- There are two types of cells:
  - Markdown and Code

Markdown cells

- Way for us to add headers / text in our notebook
- Try adding a markdown cell and entering in some text, e.g. This is my first notebook
- Shift + Enter to run the cell
- Try changing the text by adding some #s in front, e.g. ### This is my first notebook

Code cells

- Insert a code cell and type in: print('Hello world')
- We see the result below the cell
- Shift + Tab on the function, gives us help on what the function does, and the arguments it needs
  - Another way: ?print
- Ctrl + Z in the cell undo the last step

Code cells

- We can also include text/comments in a code cell (not just a markdown cell); just add a # in front, e.g.: # this function prints our inputs
- Therefore, if you wish to add in extra markdown cells, or additional comments in code cells, to clarify what the code is doing, please do!

Note

- This is not a Python coding course
- The objective is not to understand each and every single line of code, and to write out the code
- But rather to understand what a block of code in each cell, on the whole, does
- And to be able to make minor changes to the code
- Let's go through the rest of the notebook…

Library

- Think of a Python library / package as a "lego box"
- Containing many wonderful pieces that allow us to build a specific set, e.g. a plane
- In the same way, a Python library or a package contains many wonderful functions that allow us to perform a specific task, e.g. create visuals, execute ML etc
- We need to import a library / package in order to use its contents

Source: AliExpress

Numpy

- NumPy: Numerical Python, provides an efficient way to store and manipulate multi-dimensional data using a data structure known as arrays

- E.g. a matrix is 2D data, comprising of rows and columns

$$\begin{pmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{pmatrix}$$

Example of data stored in matrix

- Here, we have a 2x2 matrix storing information on students in a class at NUS

	Boy	Girl
School of Computing	15	18
Other faculties	21	16

Tensors

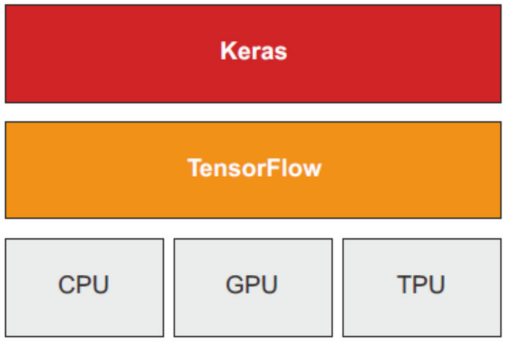
- A matrix is also known as a 2D tensor
- A vector, e.g. one containing some height info, [1.6, 1.75, 1.9, 1.55, 1.82] is a 1D tensor
- And of course, there are higher dimensional tensors too, depending on our data, e.g. a 3D tensor containing info on gender, faculty and height for each student

TensorFlow

- TensorFlow is a Python-based ML platform, developed primarily by Google
- Aim is to facilitate mathematical expressions over numerical tensors
- Advantages over NumPy include being able to run on GPUs and TPUs and to distribute the computation across many machines

Keras

- Deep learning API (application programming interface) for Python, built on top of TensorFlow
- Provides a convenient way to define and train deep learning models



Deep learning development: layers, models, optimizers, losses, metrics...

Tensor manipulation infrastructure: tensors, variables, automatic differentiation, distribution...

Hardware: execution

Source: Chollet

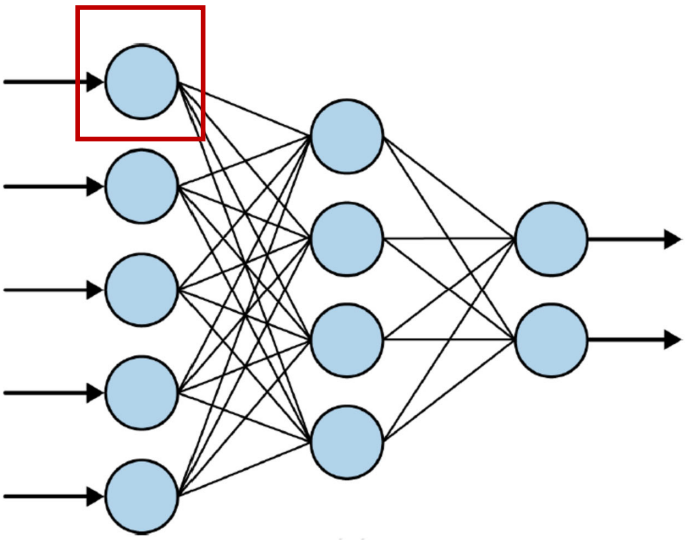
Let's go back to our notebooks

What is the actual relationship between Y and X?

- Share your answers
- What is your approach for arriving at the answer?
- Not all of us will go about arriving at the actual relationship in the same way, but irregardless, that's our learning process in action!
- And in its own way, the model is learning too
- Let's go back to our notebooks

This is the model that we have built

- The simplest neural network – just one neuron!



The loss function

- Now, the model doesn't yet know what the actual relationship between X and Y is, i.e. it doesn't know what m and c are, in Y = mX + c
- So it starts with a guess, say Y = 5X + 5
- But how good is this guess? That's where the loss function comes in

The loss function

- We already know what the answers (values of Y) are, when X = -2, -1, 0, 1, 2, 3
- Therefore, the loss function can compare these actual values, to the corresponding ones from the guessed relationship
- So when X = 0, the actual Y is -2, but the guessed Y is 5(0) + 5 = 5, so the mean squared error (our loss function) will yield 49

The optimizer function

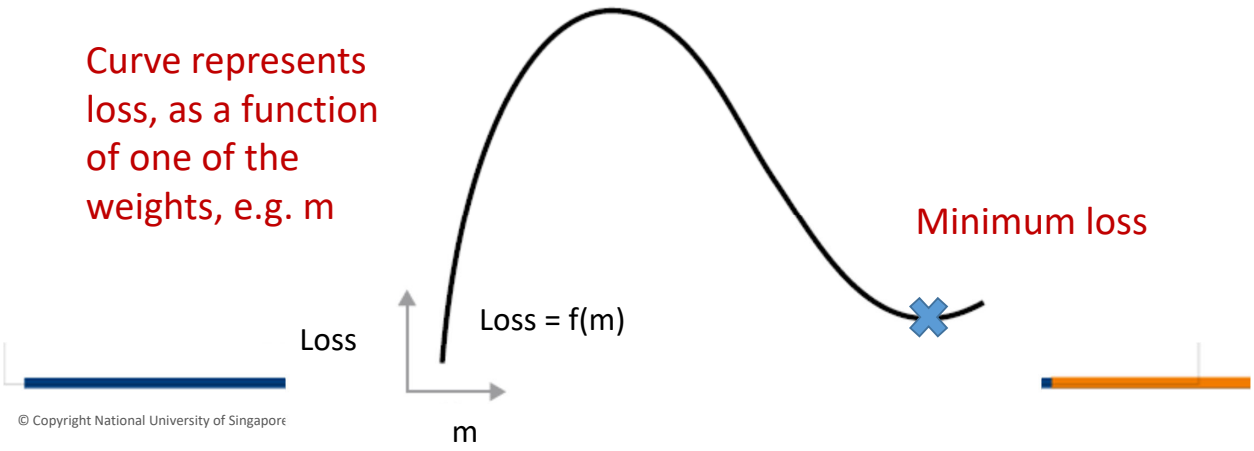
- With this knowledge, the model can make another guess – but how to guess? At random? No, we make an informed guess, using the optimizer function
- This will involve quite a bit of calculus, but TensorFlow abstracts all these calculations for us
- We just need to pick an optimizer, which in this case is stochastic gradient descent
- The job of the optimizer is to minimise the loss, and thus bring the guess closer and closer to the actual relationship

Gradient-based optimization

- In our example, the unknown parameters m and c are attributes of the layer, aka “weights” or “trainable parameters” of the layer (comprising just one neuron)
- These weights represent what the model has learned from exposure to the training data
- At the start, they are assigned random values (in our case, 5 and 5); this is known as random initialization
- From these starting values, we adjust the weights based on the loss

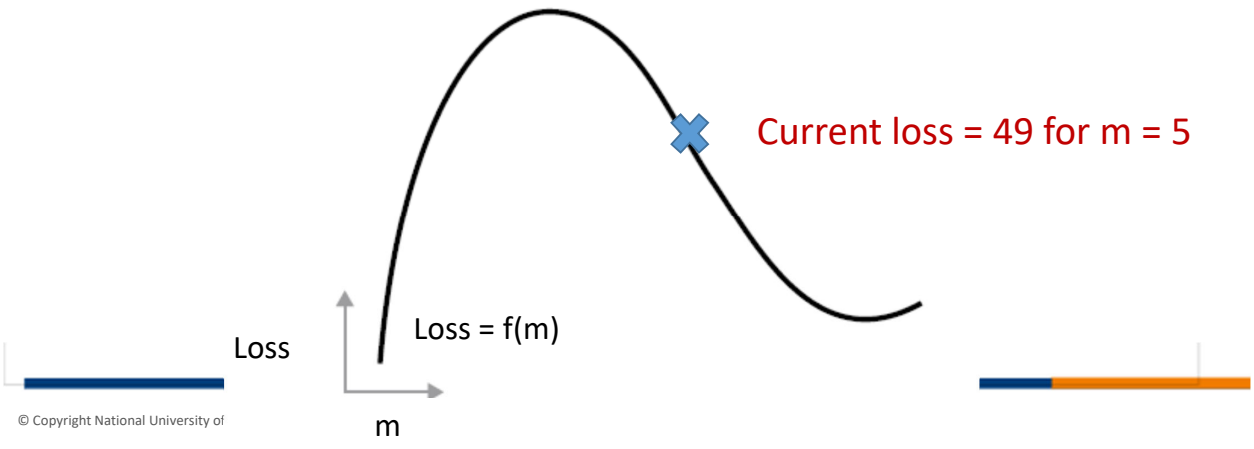
This adjustment is done using gradient-based optimization

- We are eventually trying to find the set of weights (x in the graph below) that will minimise the loss (y in the graph below)



Gradient-based optimization

- Say, currently we compute a loss based on our initial set of values (in our case, loss = 49 for m = 5)
- How shall we adjust the weight so that the loss is lower than what we have?

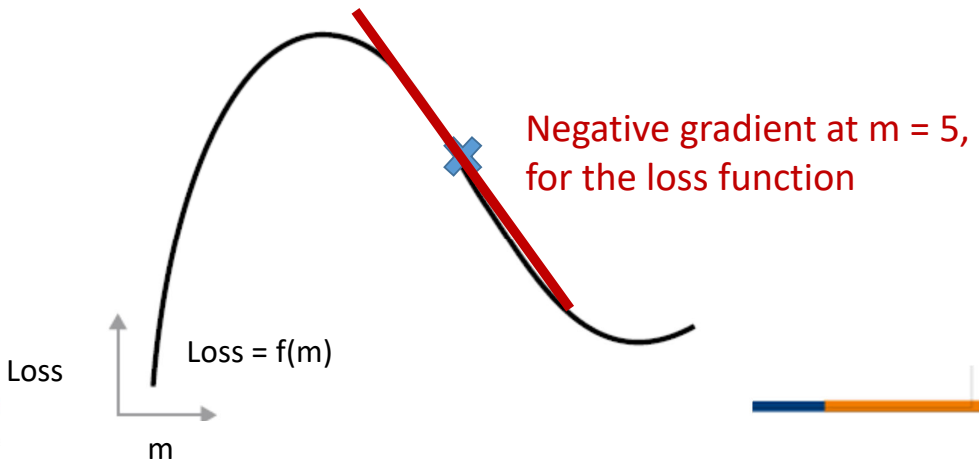


Gradient-based optimization

- One approach would be to try changing the weights' values randomly in one direction, then the next, and computing the loss each time
- But across many layers and neurons, this is extremely computationally intensive and inefficient
- There is a more effective way: gradient descent – optimization technique that powers modern neural networks

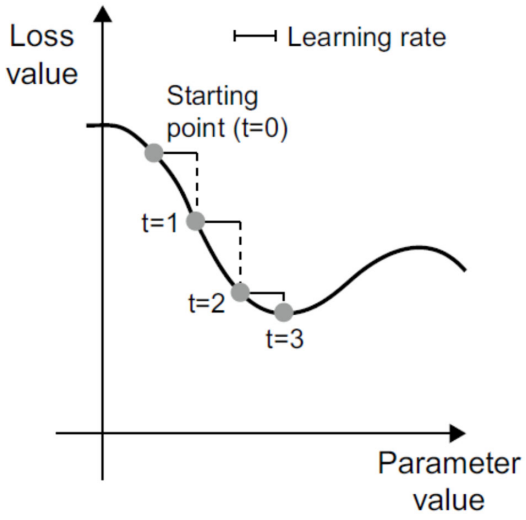
Gradient-based optimization

- We differentiate the loss function, and find its value (i.e. the gradient) with respect to each weight – we see it's negative below for this weight



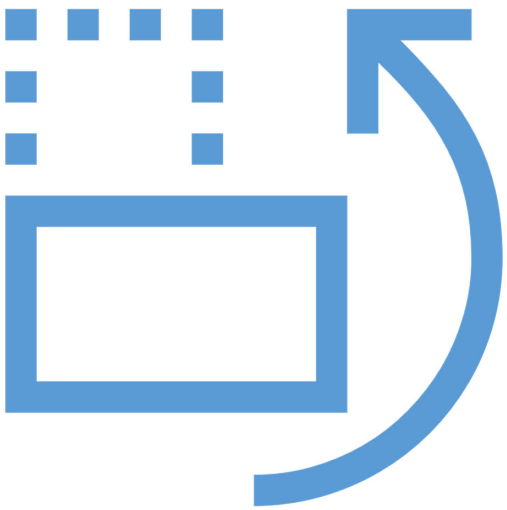
Gradient-based optimization

- Hence, to move towards the minimum loss, we need to increase the value of the weight
- Vice versa, if gradient is positive, we need to decrease the value of the weight
- In our example, because the gradient is negative, we need to increase m from 5 (just illustrative)
- How much to move down is determined both by the gradient, and by our learning rate



Gradient based optimization

- What happens if the learning rate (step size) is too small? What problem might we foresee?
- What if the learning rate is too high?



Stochastic gradient descent

- So now we know how to change the weight so that we have a small improvement in the loss – we repeat this for the other weights, in this case for c
- The exact rules governing the specific use of gradient descent are defined by the optimizer that we choose – what we have just described is the "sgd" optimizer in our example

Other optimizers

- Other optimizers are available, variants on SGD, such as Adagrad, RMSprop, etc
- These take into account previous weight updates, when computing the next weight update

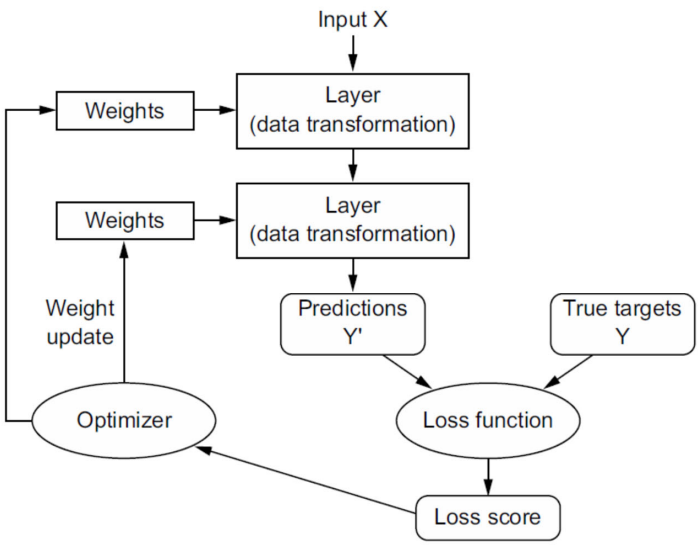
In general

- Since initial weight values are randomly assigned, we expect the loss score will be high
- But with every example data point the network processes, the weights are adjusted a little in the right direction, and the loss score drops

In general

- Aforementioned steps form our training loop
- When repeated, typically tens of iterations over thousands of data points, we will have weights that minimises the loss function

Let's return to our notebook and start the training!



Displayed results

- We have information on

Epoch number

The number of batches into which our training data is divided; here we have one batch of six inputs

The total time taken for that step to be completed (forward + backward passes)

The loss for this epoch, based on adjusted parameters from the previous epoch via gradient descent

```
Epoch 1/500
1/1 [==============================] - 0s 465ms/step - loss: 3.0479
Epoch 2/500
1/1 [==============================] - 0s 7ms/step - loss: 3.0254
Epoch 3/500
1/1 [==============================] - 0s 10ms/step - loss: 3.0029
Epoch 4/500
1/1 [==============================] - 0s 7ms/step - loss: 2.9847
Epoch 5/500
1/1 [==============================] - 0s 8ms/step - loss: 2.9700
```


Results discussion

- Is the predicted result from the model exactly what we expected?
- If not, why do you think there is a discrepancy?
- Let's go back to our notebooks to see what are the exact weights

Tracking what happens in each epoch

- In epoch #1, what is the loss computed from the forward pass?
- What are the adjusted weights in epoch #1 generated from the backward pass?
- Apply the weights across our input xs
- [-2.0, -1.0, 0.0, 1.0, 2.0, 3.0]
- Fill in the table on the next slide!

Tracking what happens in each epoch

xs	Actual ys	Predicted ys	Absolute error
-2	-8		
-1	-5		
0	-2		
1	1		
2	4		
3	7		

Mean absolute error = ?

Tracking what happens in each epoch

- The result is the loss reported in epoch #2, i.e. using the adjusted weights from epoch #1 to generate the new loss forms our forward pass for epoch #2
- Now, the backward pass of epoch #2 will generate new adjusted weights, which can then be used to calculate the loss in epoch #3, and so on
- So each epoch, per what we described earlier in our class, will have one forward and one backward pass

Recap

- Introduction to AI and Deep Learning
- Building our very first neural network
- How neural networks work

Up next

Deep learning for natural language processing

End Day 1